Mantiq — Product Requirements Document

AI-powered industrial maintenance memory platform

Field	Details
Product	Mantiq
Document type	Product Requirements Document (PRD)
Version	1.1 (renamed from EchoTrace v1.0; provider details verified)
Status	In implementation (backend complete, frontend pending)
Prepared	October 16, 2026 · Updated October 28, 2026
Owner	Shourya Singh (project lead / product owner)
Programme	Gnani AI developer programme
Submission deadline	November 10, 2026 (no extension)
Core models	Gnani Prisma (speech-to-text), Gnani Evon (reasoning); Gnani Timbre (text-to-speech, optional)
Repository	https://github.com/sourya2704/MaintiQ
Primary audience	Product owner, engineers, AI implementers, project evaluators

Purpose: define the product problem, target users, scope, workflows, AI behaviour, system requirements, data model, security, acceptance criteria and delivery plan.

Changes in v1.1

- Product renamed from **EchoTrace** to **Mantiq**.
- Gnani provider endpoints verified against official documentation (section 9).
- Processing, review and resolution states finalised as three separate fields (section 11.1).
- API list extended with routes added during implementation (section 11).
- Open questions updated with what has been resolved (section 21).
- Implementation status added (section 23).

Contents

1. Executive summary
2. Product vision and principles
3. Problem statement and opportunity
4. Goals, non-goals and scope

5. Users, personas and jobs to be done
6. User journeys
7. Functional requirements
8. Failure Memory Engine
9. AI behaviour and model integration requirements
10. Data model
11. API requirements
12. UX and UI requirements
13. Non-functional requirements
14. Error handling and recovery
15. Analytics and success metrics
16. Acceptance criteria
17. Demo dataset and test scenarios
18. Security, privacy and safety
19. Delivery plan
20. Risks and mitigations
21. Open questions and assumptions
22. Definition of done and submission checklist
23. Implementation status

1. Executive Summary

Mantiq is an AI-assisted industrial maintenance memory platform. It helps maintenance teams turn technicians' spoken observations into structured, searchable incident records, and retrieve relevant historical incidents when similar problems occur.

A technician records or uploads a voice note about a machine fault. Gnani Prisma transcribes the audio. Gnani Evon extracts structured incident details and can reason over a limited set of historical records retrieved by the application. Mantiq stores machine, incident, action and outcome information in a relational database, then surfaces related historical incidents with links to the source evidence.

The main differentiator is the **Failure Memory Engine**: a searchable connection between machine identity, symptoms, observations, actions taken and verified maintenance outcomes. Mantiq is a decision-support tool. It does not autonomously diagnose, control or authorise work on industrial equipment.

Product summary

Question	Answer
What is it?	A voice-first, AI-assisted maintenance history and incident retrieval platform.
Who is it for?	Maintenance technicians and supervisors in factories and industrial service teams.
What problem does it solve?	Maintenance knowledge is fragmented, hard to search and often lost between shifts or staff changes.

Question	Answer
What makes it different?	It links new reports to the organisation's own previous incidents and their recorded outcomes.
What is the MVP?	Machine records, Prisma transcription, Evon extraction, human review, incident history, related-incident search and outcome capture.
What is optional?	Timbre spoken summaries, advanced semantic search, exports and enterprise integrations.

2. Product Vision and Principles

2.1 Vision

Help industrial maintenance teams preserve practical knowledge and reuse it safely, so each incident makes future maintenance investigations better informed.

2.2 Product principles

- **Evidence before assertion:** show the source transcript and historical records behind suggestions.
- **Separate facts from hypotheses:** reported observations, AI-suggested causes and technician-verified outcomes stay distinct.
- **Human ownership:** technicians review and confirm records; AI does not authorise repairs.
- **Useful before impressive:** prioritise the complete voice → record → history workflow over decorative AI features.
- **Graceful degradation:** keep transcripts and user-entered information when a model call fails.
- **Integration transparency:** clearly distinguish real provider responses from mocks and sample data.
- **Privacy by design:** minimise stored audio and sensitive data, and enforce access controls.

2.3 Product positioning

Mantiq is not a generic chatbot, a replacement for a computerised maintenance management system (CMMS), or an autonomous predictive-maintenance system. It is a focused layer that makes informal maintenance observations easier to capture and existing incident knowledge easier to reuse.

3. Problem Statement and Opportunity

Industrial maintenance teams face recurring issues such as abnormal vibration, overheating, unusual noise, component wear and unexpected stoppages. Observations and repair details are spread across notebooks, spreadsheets, messaging apps and individual technicians' memories. When a similar issue occurs, staff struggle to find earlier incidents, understand which checks were performed, or tell whether an earlier repair actually resolved the problem.

Traditional maintenance tools store work orders but rely on manual data entry, and rarely turn informal spoken observations into searchable, reusable knowledge. Mantiq reduces the effort of capturing incidents and connects new observations with previous machine history.

3.1 Problem hypotheses to validate

- Technicians can capture a useful first draft faster by speaking than by filling in a long form.
- Maintenance teams need a better way to retrieve similar past incidents and their outcomes.
- Historical records become more useful when symptoms, actions and outcomes are stored separately.
- A transparent, evidence-linked suggestion is more trustworthy than a free-form AI answer.

These are hypotheses, not proven customer outcomes. Validate them through interviews or feedback from maintenance professionals where possible.

3.2 Example problem

A technician reports that motor M-104 vibrates more than usual and runs hotter. The belt was inspected but the vibration persisted. Mantiq records these details, searches M-104's history and surfaces an earlier incident if one exists. The technician can inspect the previous record and its recorded outcome, then decide what to do according to approved site procedures.

The system must not assume the earlier cause applies to the current incident. A related record is evidence to review, not proof of a diagnosis.

4. Goals, Non-Goals and Scope

4.1 MVP goals

- Create and manage machine records.
- Accept a voice recording and transcribe it with Prisma when valid credentials and API access are available.
- Generate a structured incident draft with Evon.
- Let technicians review, edit and confirm drafts.
- Store incidents and maintenance outcomes in PostgreSQL.
- Retrieve related historical incidents and show why each result is relevant.
- Provide a professional dashboard and searchable machine/incident history.
- Handle provider failures without losing transcripts or falsely reporting success.

4.2 Non-goals for the MVP

- Autonomous machine diagnosis or root-cause confirmation.
- Direct control of machinery, PLCs, sensors or safety systems.
- Predictive maintenance claims based on telemetry the product does not collect.
- Replacing an existing CMMS/EAM platform or integrating with every enterprise system.
- Guaranteed support for every language, accent, audio format or industrial domain.
- Fine-tuning a model or building custom speech recognition.

- Fleet-wide predictive analytics before core incident history and retrieval work.

4.3 Priorities

Priority	Meaning	Examples
P0 — Must have	Required for MVP acceptance	Prisma, Evon extraction, saved incidents, historical retrieval, review, access control
P1 — Should have	Important if time allows	Audit history, export, improved search, manual transcript fallback
P2 — Could have	Optional after MVP stability	Timbre playback, vector search, enterprise integrations
Out of scope	Not part of this submission	Autonomous controls, guaranteed diagnosis, unsupported telemetry features

5. Users, Personas and Jobs to Be Done

Persona	Needs	Key jobs
Maintenance technician	Fast incident capture, useful history, editable records	Report a fault, review a transcript, inspect related incidents, record a repair outcome
Maintenance supervisor	Visibility into open issues, recurring symptoms and record quality	Review incidents, follow unresolved issues, inspect machine history, verify outcomes
Administrator	Safe access and configuration	Manage users and configuration without exposing provider secrets
Developer / evaluator	Reproducible proof of working integrations	Run the demo, inspect integration status, understand limitations

5.1 Primary user

A maintenance technician near equipment, with little time for data entry, who needs a quick way to preserve observations. The interface must work well on desktop and tablet and must not assume the technician wants to chat with an AI.

5.2 Secondary user

A supervisor who needs to understand unresolved incidents, inspect the history behind recommendations, and make sure AI-generated hypotheses never become verified maintenance records without review.

6. User Journeys

6.1 Create an incident from a voice note

1. The technician signs in and selects a machine (or creates a machine record).

2. The technician records audio in the browser or uploads an existing recording.
3. The backend validates file type, size, duration and access permissions.
4. Prisma transcribes the recording; the UI shows processing status and the returned transcript.
5. The technician reviews and can correct the transcript before analysis.
6. Evon generates a structured incident draft from the transcript and relevant machine context.
7. The backend validates the output schema and flags missing or uncertain information.
8. The Failure Memory Engine searches for relevant historical incidents.
9. The UI presents the draft and related records for review.
10. The technician edits and confirms the incident; generated fields stay marked unreviewed until confirmed.

6.2 Investigate a recurring problem

1. The technician opens a machine profile, a new incident, or the memory search.
2. The system retrieves prior incidents using machine identity and reported symptoms.
3. Results show the relevance rationale and links to the original incident.
4. The technician inspects past actions and recorded outcomes.
5. The technician decides what to do using approved procedures and their own judgement.

6.3 Record a maintenance outcome

1. After inspection or repair, the technician opens the incident.
2. They record what was inspected, actions taken, parts used (if known) and whether the issue was resolved.
3. A verified cause can be recorded if known and supported.
4. The system records who submitted the outcome and when.
5. Future searches can retrieve this record as historical evidence.

6.4 Failure and recovery journey

If Prisma or Evon fails, the user sees which step failed and can retry. The transcript and entered information remain available according to the retention policy. A retry never silently creates a duplicate incident or claims that a failed model call succeeded.

7. Functional Requirements

ID	Requirement	Priority	Acceptance-oriented description
FR-001	Authentication	P0	Users can register and sign in. Passwords are securely hashed; protected endpoints require valid authentication.
FR-002	Authorization	P0	Backend authorization prevents access to machines, incidents and audio outside the user's organisation.
FR-003	Machine management	P0	Authorised users can create, view, edit and search machines with unique asset codes and basic metadata.
FR-004	Audio intake	P0	Users can upload supported audio and see validation errors, upload progress and processing status.

ID	Requirement	Priority	Acceptance-oriented description
FR-005	Prisma transcription	P0	Backend calls the documented Prisma API and stores the actual transcript plus safe provider metadata for troubleshooting.
FR-006	Manual transcript fallback	P1	If transcription is unavailable, users may enter text manually; the record is labelled manual, not Prisma-generated.
FR-007	Transcript review	P0	Users can review and edit the transcript before analysis, keeping both original and edited text.
FR-008	Evon extraction	P0	Backend obtains a structured incident draft from Evon and validates it against a defined schema.
FR-009	Uncertainty handling	P0	System separates reported observations, suspected causes, missing information and verified outcomes.
FR-010	Incident review	P0	Technicians can edit, save and confirm extracted fields; review status and timestamps are recorded.
FR-011	Historical retrieval	P0	System searches stored incidents and shows relevant records with links, dates, machine IDs, actions and outcomes.
FR-012	Retrieval explanation	P0	Each related result has a simple rationale such as same machine, symptom match or text relevance.
FR-013	Outcome capture	P0	Technicians can record actions, resolution status, verified cause if known, and verifier/time metadata.
FR-014	Machine timeline	P0	Machine details show its incidents chronologically.
FR-015	Search and filters	P0	Users can search/filter by machine, date, status and symptom/keyword with pagination.
FR-016	Dashboard	P0	Dashboard metrics come from real database records and include useful empty states.
FR-017	Processing states	P0	AI processing has explicit states and visible failures; retries never silently create duplicates.
FR-018	Integration transparency	P0	Configuration and logs distinguish live provider responses from mocks and seeded data.
FR-019	Text-to-speech	P2	If available, Timbre reads a generated maintenance summary aloud.
FR-020	Export	P1	Users can export an incident summary in a downloadable format (Markdown, JSON).
FR-021	Audit history	P1	Important edits, review events and outcome updates record actor and timestamp.
FR-022	Retention/deletion	P0	Authorised users can apply configured audio retention/deletion rules.

8. Failure Memory Engine

The Failure Memory Engine is Mantiq's main differentiator. It uses only incidents stored in Mantiq, never history invented by a language model.

8.1 Retrieval sequence

1. Search incidents for the exact machine first.
2. Search other machines of the same type.
3. Use PostgreSQL full-text search, canonical symptom tags and metadata filters (MVP).
4. Optionally add vector similarity later if it measurably improves retrieval.
5. Rank candidates with transparent signals, in this order: same machine → same machine type → number of shared symptoms → text relevance.
6. Return source incident IDs and a deterministic explanation of why each result was included.
7. Return only records the current user is authorised to access.
8. Index facts only (transcript, summary, symptoms, observations, actions, verified outcomes). Suspected causes and suggested checks are **not** indexed, so hypotheses never drive retrieval.

8.2 Retrieval display

- Machine asset code and machine type.
- Incident date and short summary.
- Symptoms and observations recorded at the time.
- Actions taken and outcome, or "unknown/unrecorded".
- Relevance rationale (e.g. "Same machine (M-104)", "Shared symptom: vibration").
- Link to open the original incident.

8.3 Safeguards

- A similar incident is not proof of the current cause.
- Suspected causes are never presented as verified outcomes.
- No fabricated similarity scores, and none presented as diagnostic confidence.
- Always show the date and machine identity for historical records.
- If no relevant records are found, show an honest no-match state.
- Never include unrelated incidents just to fill the list. A different machine type needs a shared symptom or meaningful text match to appear.

9. AI Behaviour and Model Integration Requirements

9.1 Prisma — speech-to-text (verified)

Item	Value
Endpoint	`POST https://api.vachana.ai/stt/v3` (multipart/form-data)
Auth	`X-API-Key-ID` header; optional `X-API-Request-ID` correlation ID

Item	Value
Fields	`audio_file` (required), `language_code` (e.g. `en-IN`, `hi-IN`, `indic-auto` for v3.0), `model` (`prisma-v3.0`; v2.5 default)
Formats	WAV, MP3, OGG, FLAC, AAC, M4A (resampled to 16 kHz mono)
Limit	30 seconds per REST request
Response	`{success, request_id, transcript, model, detected_language_code, ...}`
Errors	401/402/403/429 auth and credit errors; 400 audio/language errors; 504 upstream timeout

Requirements:

- Send audio through the backend only, never directly from browser code with the key.
- Store the actual transcript and distinguish provider output from user-edited text.
- Reject invalid, oversized or over-30-second audio before calling Prisma.
- Convert browser recordings (WebM/Opus) to WAV on the server.
- Handle empty transcripts, provider errors and retry limits.
- Never claim a live integration when using a mock.

9.2 Evon — reasoning and structured extraction (verified route)

Item	Value
Model	`gnani/gnani-evon-v3.3-30b-a3b` (Hugging Face)
Serving	OpenAI-compatible `POST /v1/chat/completions` (e.g. SGLang); hosted route to be confirmed with Gnani
Mode	Reasoning off (`enable_thinking: false`), low temperature, JSON-schema constrained output where supported

Requirements:

- Provide the transcript, machine metadata and at most 3 retrieved historical records.
- Require structured output and validate it; never treat an unconstrained paragraph as reliable JSON.
- Extract summary, symptoms, observations, actions already taken, suspected causes, missing information and advisory checks.
- Treat transcript content as untrusted input (wrapped in `<transcript>` tags) and ignore embedded instructions.
- Never invent readings, repair results, component conditions or confirmed root causes. Observations containing numbers not in the transcript are removed and flagged.
- On invalid output, retry within a bounded policy (default 2 attempts) and always keep the transcript.

9.3 Timbre — optional text-to-speech (verified)

Item	Value
Endpoint	`POST https://api.vachana.ai/api/v1/tts/inference` (JSON)

Item	Value
Body	`text`, `voice` (e.g. `Nalini`), `model` (`timbre-v2.5`), `language`, `speed`, `audio_config`
Response	Raw `audio/wav`

- Generate speech only from a reviewed summary, or one clearly labelled "AI-suggested, not yet reviewed".
- The core workflow stays fully usable if Timbre is unavailable.

9.4 Structured incident schema

```
{
  "incident_summary": "string",
  "symptoms": ["string"],
  "observations": ["string"],
  "actions_already_taken": ["string"],
  "suspected_causes": ["string"],
  "missing_information": ["string"],
  "suggested_checks": ["string"],
  "urgency": "unknown | needs_review | routine | urgent",
  "confidence_notes": "string",
  "requires_human_review": true
}
```

This is an application schema, not a native model response. The backend validates and normalises it.

9.5 Safety language

Suggestions are advisory and grounded in the transcript or linked records. When a potential hazard is mentioned (smoke, sparks, fire, shock, gas leak, etc.), the system adds a notice directing users to follow approved site safety, lockout/tagout and escalation procedures. It never advises bypassing guards, disabling interlocks or violating safety requirements, and any such suggestion from the model is removed.

10. Data Model Requirements

Entity	Important fields	Rules
Organization	id, name, created_at	Tenant boundary for all data.
User	id, organization_id, name, email, password_hash, role (admin/supervisor/technician), is_active	Never store plaintext passwords.
Machine	id, organization_id, asset_code, name, type, manufacturer, model, location, status, timestamps	Asset code unique within an organisation.

Entity	Important fields	Rules
Incident	id, organization_id, machine_id, reporter_id, transcript_original, transcript_edited, transcript_source, summary, symptoms, symptom_tags, observations, actions_taken, suspected_causes, missing_information, suggested_checks, urgency, processing_status, review_status, resolution_status, search_text, search_vector, timestamps	Original and edited transcript both kept. Three separate status fields.
MaintenanceOutcome	id, incident_id, verified_cause, action_performed, parts_used, resolution_status, notes, verified_by, resolved_at	Verified outcome kept apart from model hypotheses.
AudioAsset	id, incident_id, storage_key, mime_type, size_bytes, duration, retention_status, delete_after	Metadata and storage reference only; no binary in incident rows.
AIProcessingJob	id, incident_id, provider, model, mode (live/mock), status, attempts, provider_request_id, latency_ms, error_code, safe_error_message	Never logs secrets or full audio/transcripts.
AuditEvent	id, organization_id, actor_id, entity_type, entity_id, action, details, created_at	Records who changed what and when.
RelatedIncident	—	Computed dynamically, not persisted.

10.1 Data integrity

- Foreign keys, indexes (including GIN on the search vector and symptom tags), migrations and timestamps.
- Explicit deletion behaviour for related audio and incident records.
- One source of truth for confirmed maintenance outcomes.
- JSONB only where fields are genuinely flexible (audit details).
- Record who reviewed or changed important fields.

11. API Requirements

Method	Endpoint	Purpose
POST	/api/auth/register	Register organisation and admin account
POST	/api/auth/login	Authenticate user
GET	/api/auth/me	Current user
GET/POST	/api/users	List / create users (admin)
PATCH	/api/users/{id}	Update role or active state (admin)
GET	/api/machines	List/search accessible machines

Method	Endpoint	Purpose
POST	/api/machines	Create machine
GET	/api/machines/types	Distinct machine types
GET	/api/machines/{id}	Machine details and timeline
PATCH	/api/machines/{id}	Edit machine metadata
DELETE	/api/machines/{id}	Delete a machine with no history (confirm required)
POST	/api/incidents	Create incident from audio or manual transcript (Idempotency-Key supported)
GET	/api/incidents	List/search/filter incidents
GET	/api/incidents/{id}	Incident details with provenance labels
PATCH	/api/incidents/{id}	Edit transcript / review fields
DELETE	/api/incidents/{id}	Delete incident (supervisor/admin, confirm required)
POST	/api/incidents/{id}/transcribe	Retry Prisma transcription
POST	/api/incidents/{id}/analyze	Run/retry Evon structured analysis
POST	/api/incidents/{id}/confirm	Technician confirmation
GET	/api/incidents/{id}/jobs	AI processing job history
GET	/api/incidents/{id}/related	Related historical incidents
POST	/api/incidents/{id}/outcome	Record maintenance outcome
GET	/api/incidents/{id}/outcomes	Outcome history
GET	/api/incidents/{id}/audit	Audit trail
GET/DELETE	/api/incidents/{id}/audio/{audio_id}	Play / delete recording
GET	/api/incidents/{id}/export	Export Markdown or JSON
POST	/api/incidents/{id}/speak	Optional Timbre summary
GET	/api/memory/search	Search history before creating an incident
GET	/api/memory/symptom-tags	Canonical symptom vocabulary
GET	/api/dashboard/summary	Dashboard metrics
GET	/api/settings/integrations	Live vs mock status (no secrets)
POST	/api/admin/retention/run	Apply audio retention policy
GET	/api/health	Health check

All routes use a consistent error envelope `{"error": {"code", "message", "details"}}` and enforce authorisation and validation.

11.1 Processing, review and resolution states

These are three **separate** fields so AI status, human review and repair status are never mixed.

Field	States
processing_status	uploaded → transcribing → transcribed / transcription_failed → analyzing → awaiting_review / analysis_failed
review_status	draft → confirmed
resolution_status	open → monitoring / resolved / unresolved

12. UX and UI Requirements

12.1 Visual direction

- Professional industrial SaaS design, not a chatbot-first layout.
- Neutral content area with a restrained dark slate sidebar and one industrial accent colour.
- Consistent typography, spacing and reusable components.
- Accessible labels, keyboard navigation, visible focus states and readable contrast.
- Useful loading, empty, processing, error and success states.
- No decorative charts or hardcoded operational statistics.
- Responsive on desktop and tablet; core workflows usable on smaller screens.

12.2 Required pages

Page	Main content
Login / registration	Authentication forms, validation and clear error feedback
Dashboard	Real counts, recent incidents, unresolved issues and machine activity
Machine list	Search, filters, asset identifiers and status
Machine detail	Metadata, chronological incident timeline and related records
New incident	Machine selector, recorder/upload, file status and transcript review
Incident review	Editable extraction, uncertainty indicators and confirmation action
Incident detail	Transcript, observations, hypotheses, linked history, actions and outcome
Related incidents	Ranked historical results with relevance rationale and source links
Outcome entry	Repair actions, verified cause if known, resolution status and notes
Settings / integration status	Non-secret configuration and clear mock-versus-live status

12.3 Interaction details

- Disable duplicate submits while an identical request is processing.
- Show progress and which stage is running.
- Keep the transcript accessible if Evon fails.
- Make source records clickable, with an easy way back to the current incident.
- Use explicit labels such as "AI-suggested" and "Technician-confirmed".
- Require confirmation before destructive actions such as deleting audio or an incident.

13. Non-Functional Requirements

Category	Requirement
Security	Secure password hashing, protected endpoints, server-side authorisation, secret management, safe upload handling, tenant isolation
Reliability	Model failures never erase user data; retries are visible and safe
Performance	Paginated lists; long model calls run as background jobs
Maintainability	Separate provider adapters, business logic, routes, schemas and database models
Observability	Log processing state, latency and safe error details, never API keys
Privacy	Minimise audio retention, support deletion, disclose external AI processing
Accessibility	Keyboard-operable controls, labels, readable contrast, status announcements
Portability	Documented local setup and environment configuration; no hardcoded deployment URLs
Cost control	Track usage where available, limit retries, use short test recordings

14. Error Handling and Recovery

Scenario	Expected behaviour
Invalid, oversized or >30 s audio	Reject before calling Prisma and explain the requirements
Prisma failure	Mark transcription failed; keep the recording per policy; offer retry and manual entry
Unclear transcript	Show it for correction; never silently change the facts
Malformed Evon output	Validate; retry within the limit; otherwise show a recoverable failure and keep the transcript
No historical match	Show an honest no-match state and allow incident creation
Database write failure	Don't show success; keep form data where safe; allow retry
Unauthorised access	Return a safe 404/403 without leaking record data
Timbre unavailable	All core workflows keep working
Duplicate retry	Row locks, job-state checks and Idempotency-Key prevent duplicate records and charges
Job stuck after a crash	Jobs running longer than 5 minutes are marked failed and can be retried

15. Analytics and Success Metrics

Metric	Evaluation method
Transcription quality	Word error rate or key-detail preservation on a reviewed test set

Metric	Evaluation method
Extraction accuracy	Whether important extracted fields match known facts in test transcripts
Retrieval relevance	Manual judgement of whether top results are genuinely relevant
Workflow completion	Test scenarios completing upload → transcription → review → save → retrieval
Processing latency	End-to-end and provider latency (recorded per job)
Cost per recording	Provider usage/cost where available
Review effort	Qualitative feedback on how much editing drafts need
User trust	Whether evidence, uncertainty and verified outcomes are easy to tell apart

Only measured results are reported. No claims of reduced downtime, improved safety or cost savings without real-world evidence.

16. Acceptance Criteria

1. An authorised user can sign in and create a machine record.
2. A valid recording can be submitted and transcribed by the real Prisma integration when credentials are configured.
3. The transcript can be reviewed and edited without losing provenance.
4. Evon produces a structured draft that passes backend validation, or the app shows a clear recoverable failure.
5. AI hypotheses are visibly separated from observations and verified outcomes.
6. The user can save and reopen an incident.
7. A later incident retrieves a seeded relevant historical record through real database search when the test data supports a match.
8. Each related result links to its source and explains why it matched.
9. A technician can record a maintenance outcome separately from AI suggestions.
10. Dashboard figures reflect database records, not hardcoded values.
11. Unauthorised users cannot access records outside their organisation.
12. Provider failures and retries never silently lose transcripts or create duplicate incidents.
13. The README explains setup, environment variables, migrations, seed data and model configuration.
14. The demo clearly identifies mocked integrations and never presents them as live calls.

17. Demo Dataset and Test Scenarios

A small, clearly labelled **fictional** dataset: one demo organisation, three users (admin, supervisor, technician), five machines (two electric motors, a centrifugal pump, a conveyor, an air compressor) and seven incidents covering repeated symptoms, different outcomes, an unresolved case, a case with no recorded outcome and a draft awaiting review.

Scenario	Expected result
Motor vibration plus elevated temperature	Transcript and extraction keep both symptoms and the unsuccessful belt inspection
Same machine has a prior similar incident	The prior record appears first with date, actions and recorded outcome
Similar symptom on a different machine type	The result ranks lower and says "Different machine type"
No temperature measurement in transcript	Marked as missing rather than invented
Prior incident has no verified outcome	UI shows "unknown/unrecorded"
Model call fails	Transcript stays available and retry is visible
Cross-organisation access attempt	Backend denies access

18. Security, Privacy and Industrial Safety

18.1 Security

- Gnani credentials live only in backend environment variables or a secrets manager.
- Real keys are never committed or placed in browser bundles.
- File type, size and content are validated; storage names are generated safely.
- Authorisation is enforced in every relevant backend route.
- Authentication endpoints are rate-limited.
- Passwords, API keys, full audio and unnecessary transcript content are never logged.
- Production startup is refused with a weak JWT secret.

18.2 Privacy and retention

- Only authorised recordings are used for testing and demos.
- Audio retention is configurable (default 30 days); deletion keeps the transcript and incident.
- Audio is served only through authenticated, organisation-scoped endpoints, never public URLs.
- External AI processing of audio/transcripts by Gnani services is disclosed.
- Fictional sample records are used when real industrial data is unavailable.

18.3 Industrial safety

- Mantiq is decision support, not an autonomous maintenance system.
- The MVP is never connected to machine controls or safety systems.
- It never recommends disabling guards, bypassing interlocks or ignoring site procedures.
- AI suggestions stay unverified until a qualified human reviews them.
- Source evidence and uncertainty are preserved in every incident record.

19. Delivery Plan

Dates	Milestone	Exit condition	Status
Oct 16–19, 2026	Foundation and provider verification	Repository, database, core models, documented Prisma/Evon access path	✅ Done
Oct 20–24	Incident pipeline	Audio/transcript, structured extraction, validation, saved incidents	✅ Done (backend)
Oct 25–29	Failure Memory Engine	Historical search, linked source records, outcome capture	✅ Done (backend)
Oct 30–Nov 3	Frontend integration	Dashboard, machine pages and incident review on real endpoints	⏳ Next
Nov 4–6	Testing and deployment	Live Gnani verification, end-to-end workflow, demo environment deployed	⏳ Pending
Nov 7–9	Submission preparation	Demo recorded, write-up and social post ready, links checked	⏳ Pending
Nov 10	Programme deadline	Final submission completed	—

20. Risks and Mitigations

Risk	Mitigation
Credentials or provider docs unavailable	Endpoints verified early; clearly labelled mock modes for local development, never claimed as live
Evon inference slow or resource-heavy	Background jobs, limited context (≤3 history records), reasoning off; confirm hosted route with Gnani
Model hallucinations	Schema validation, removal of invented numbers, evidence links, labelled hypotheses, mandatory human review
Poor retrieval quality	Start with exact machine matching, symptom tags and full-text search; evaluate on the seed set
Prisma 30-second limit	Validate duration up front; guide technicians to short notes; batch/streaming APIs as a later option
Scope growth	Timbre, vector search, exports and integrations stay optional until MVP acceptance passes
No real factory data	Fictional examples; honestly state the lack of field validation
Audio privacy concerns	Authorised recordings, retention controls and data minimisation

21. Open Questions and Assumptions

21.1 Resolved

- **Prisma endpoint, auth, formats and limits:** resolved, see 9.1.

- **Evon model and serving interface:** model and OpenAI-compatible route identified, see 9.2.
- **Timbre endpoint and output format:** resolved, see 9.3.
- **Single vs multi-tenant:** multi-tenant isolation by organisation from day one.

21.2 Still open

- Is Evon available as a Gnani-hosted endpoint for programme participants, or must it be self-hosted? What are the licence and resource constraints?
- Where can programme credit usage be monitored, and how are credits consumed across models?
- Which deployment and storage services fit the budget?
- Can a maintenance professional review the workflow before submission?

21.3 Assumptions

- The project is individual-led unless team members are confirmed.
- Real credentials may arrive after initial development; mock modes cover the gap.
- The MVP uses fictional data unless authorised real data is available.
- All AI suggestions require human review.
- The deadline is November 10, 2026, with no extension.

22. Definition of Done and Submission Checklist

The Mantiq MVP is complete when an authorised user can:

- create a machine and upload an audio recording;
- obtain a real Prisma transcript;
- generate and review an Evon incident draft, then save it;
- retrieve relevant historical incidents from the database;
- inspect recorded outcomes and record a new outcome.

It must also have appropriate access controls, visible error handling, setup documentation and a reproducible demo. Timbre is optional and not required for MVP acceptance.

Programme submission checklist

- ☐ Participant workspace has the problem, solution, target users, models/APIs and member contributions.
- ☐ Working demo or demo video is ready.
- ☐ Written project explanation is complete.
- ☐ The exact Gnani models and APIs used are listed (Prisma v3.0, Evon v3.3, Timbre v2.5 if used).
- ☐ LinkedIn or X post is published and its URL available.
- ☐ All required links submitted through the programme page by November 10, 2026.

23. Implementation Status (as of October 28, 2026)

Area	Status
Backend API (FR-001 to FR-022)	✅ Implemented and tested (25 automated tests on PostgreSQL)
Prisma / Evon / Timbre adapters	✅ Implemented; mock modes tested; live calls pending credentials
Failure Memory Engine	✅ Implemented
Database schema, migrations, demo seed	✅ Implemented
Frontend	⏳ Not started
Live provider verification	⏳ Waiting on credentials
Deployment	⏳ Not started
Demo video and submission	⏳ Not started

See HLD.md for the system architecture.